



SSH

Introducción
OpenSSH
Túneles
Autenticación

SSH

SSH es un protocolo pero también puede ser visto o analizado como una herramienta para encriptar, una aplicación cliente/servidor o una interfaz de comandos.

En su forma más habitual o común SSH puede ser visto como un reemplazo seguro para Telnet, ya que tanto los comandos como los datos viajan encriptados.

Aunque SSH es mucho más potente y versátil que un mero login remoto encriptado.

SSH

El protocolo SSH ofrece: autenticación, encriptación e integridad



“soy mmoss@pampero.itba.edu.ar”

“y yo soy pampero.itba.edu.ar”



“hola mundo”

“#@%6h/&%=?¡”

“hola mundo”

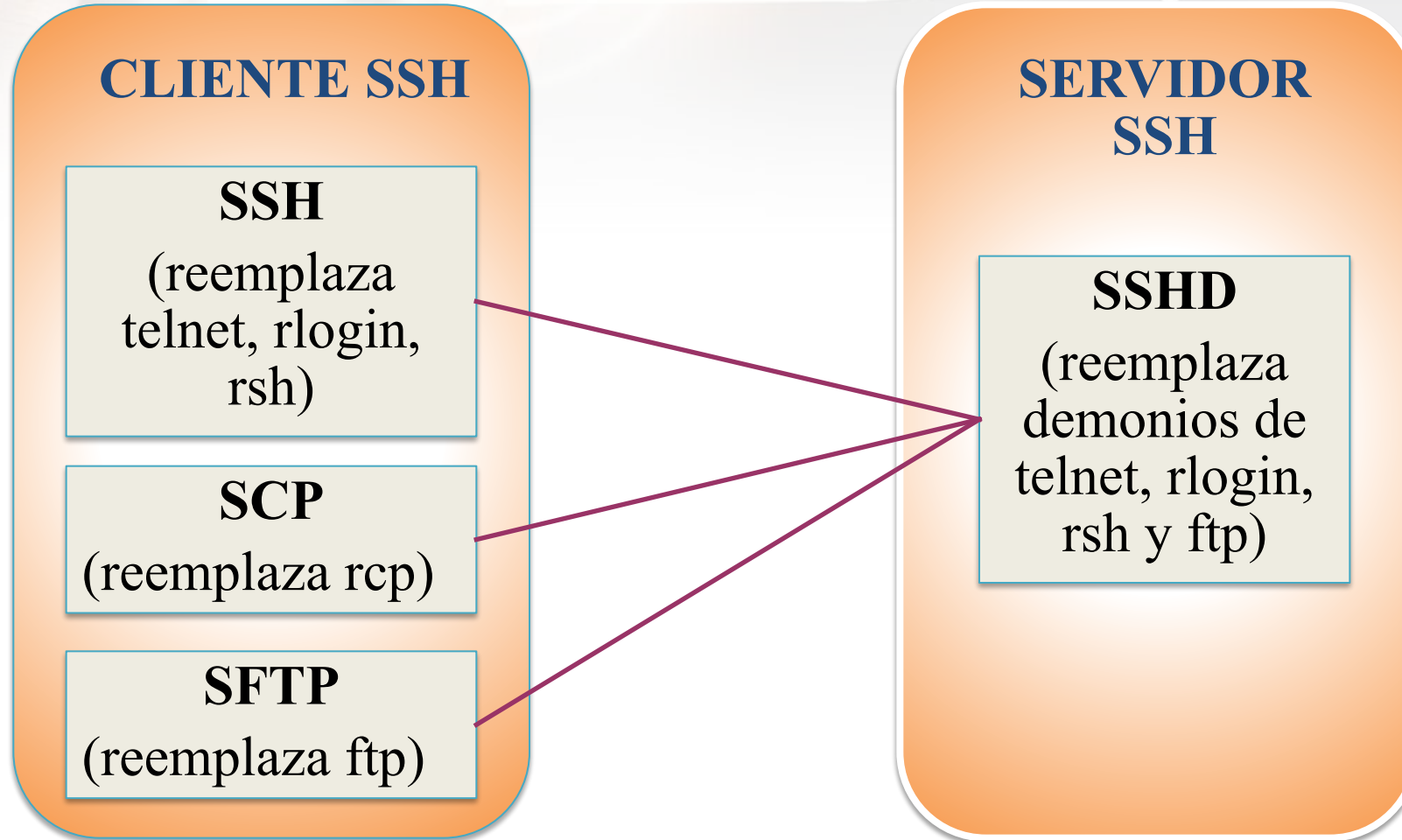
“hola mundo”

“#@%6h/XXX?¡”

¡Mensaje alterado!

SSH

SSH es el reemplazo seguro de varias aplicaciones



SSH

Los principales servidores SSH son:

- OpenSSH: Windows, Unix y Linux
- SSH Communications: Windows, Unix y Linux
- VanDyke's Vshell, BitVise: Windows.

Nosotros utilizaremos OpenSSH, que viene instalado por defecto en las distribuciones Linux.

El demonio que utiliza es **sshd** y escucha, por defecto, en el puerto 22.

OpenSSH

Componentes básicos de OpenSSH

- sshd: Es el demonio que maneja las conexiones entrantes
 - No debe ser iniciado por inetd o xinetd
- ssh: Comando que reemplaza a otros como rsh y rlogin,
 - para loguearse en forma remota o enviar un comando
 - a un servidor remoto.
- scp: Versión segura de rcp, que permite copiar archivos
- sftp: Versión segura de FTP
- ssh-keygen: generador de claves públicas y privadas.

OpenSSH

El archivo principal de configuración es `/etc/sshd_config`.
Algunas de las opciones generales son:

```
#Port 22
#Protocol 2,1
#ListenAddress 0.0.0.0
#ListenAddress ::

#LoginGraceTime 120
PermitRootLogin no
#StrictModes yes
```

Ejemplos de uso

Sesión de terminal remota

```
user@server:~$ ssh pi@pampero.itba.edu.ar
Password: *****
Last Login Wed May 4 16:12:22 from ....
Linux 2.4.25-lids1.2.0rc2
user@server:~$ ...
user@server:~$ logout
```

**La misma
interacción que con
telnet / nc**

Ejecución remota de un comando

```
user@server:~$ ssh user@host rm /tmp -r
```


Ejemplos de uso

Sesión de terminal remota

Algunas de las opciones para ssh son:

- ◆ -1 Fuerza ssh a usar SSH versión 1.
- ◆ -2 Fuerza ssh a usar SSH versión 2.
- ◆ -C Comprime los datos
- ◆ -c blowfish | 3des | des Elegir método de encriptación
- ◆ -v Muestra información de depuración
- ◆ -4 Forzar a IP v4
- ◆ -6 Usar únicamente IP v6
- ◆ -E *log_file* Info de debug a un archivo
- ◆ -p *port* Usar un puerto distinto a 22
- ◆ -N Sólo conecta, no ejecuta comando remoto

SSH Versión 1

Las primeras versiones de OpenSSH 1 contenían errores de implementación que la hacían vulnerable, como bien lo sabía Trinity

```
Port      State      Service
22/tcp    open       ssh

No exact OS matches for host

Nmap run completed -- 1 IP address (1 host up) scanned
# sshnuke 10.2.2.2 -rootpw="Z10N0101"
Connecting to 10.2.2.2:ssh ... successful.
Attempting to exploit SSHv1 CRC32 ... successful.
Resetting root password to "Z10N0101".
System open: Access Level <9>
# ssh 10.2.2.2 -l root
root@10.2.2.2's password: 
```

enter password



Ejemplos de uso

Copiar archivos

```
mgarbe@pampero:~$ scp yo@casa.no-ip.info:*.pdf .  
Password: *****  
parcial1.pdf100% 9428 9.2KB/s 00:00  
parcial2.pdf100% 8508 1.5KB/s 00:01  
final.pdf 100% 7099 1.1KB/s 00:00
```

Además de las opciones vistas para ssh, scp incluye:

- -q Deshabilita porcentaje de progreso
- -r Copia recursivamente un directorio

Ejemplos de uso

FTP seguro

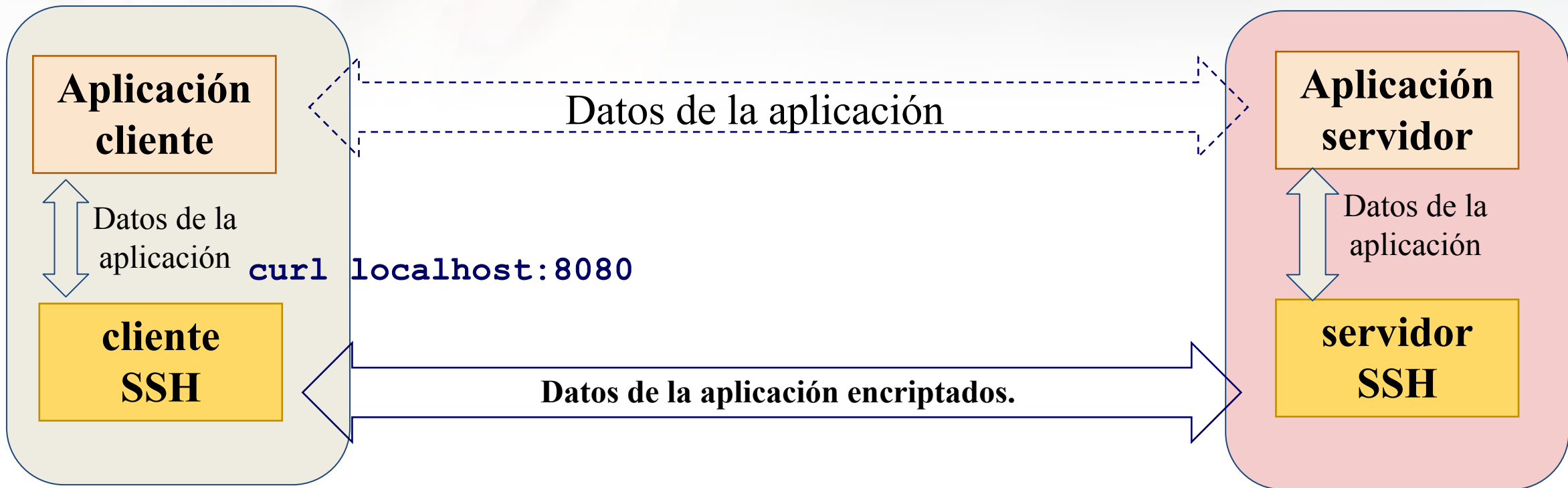
```
user@server:~$ sftp mgarbe@pampero.itba.edu.ar
Password: *****
sftp> ll
sftp> lpwd
sftp> quit
```

Alguna opción particular de sftp:

- `-B buffer_size` Buffer de transferencia.
- `-b batch_file` Lee los comandos desde un archivo y no desde stdin.

Local Port forwarding

SSH puede encriptar en forma transparente el flujo de datos de otra aplicación (*tunneling*)



```
$ ssh -L8080:webserver:80 SSHserver
```

SSH

Port forwarding

SSH port forwarding protege conexiones TCP redirigiéndolas a través de una conexión SSH. Una vez establecida la conexión se le pueden dar diversos usos, como por ejemplo:

- Acceder a servicios TCP (SMTP, IMAP, POP, LDAP, etc.) a través de un firewall que no permita conexiones a ellos.
- Protege la sesión con estos servicios, encriptando claves y datos.
- Enviar mensajes usando un servidor SMTP que no permita relaying, trabajando en otra red.

Notar que funciona sólo con TCP, no funciona con protocolos basados en UDP como DNS, DHCP, NFS, NetBIOS

Port forwarding

Local forwarding.

La sintaxis para indicar que se va a redirigir lo enviado a un puerto local a otro puerto en otro host es

```
$ ssh -Lport:host:hostport SSHserver
```

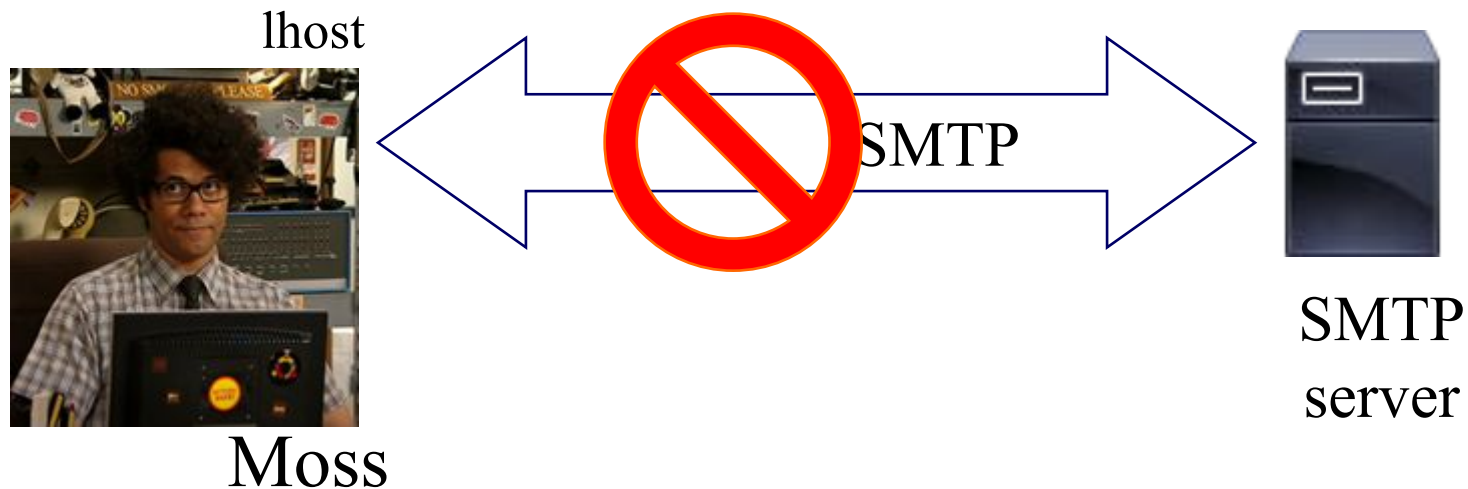
puerto del socket pasivo local

Host remoto, visible desde el servidor SSH

Puerto de la aplicación en el host remoto

Local Port forwarding: ejemplo

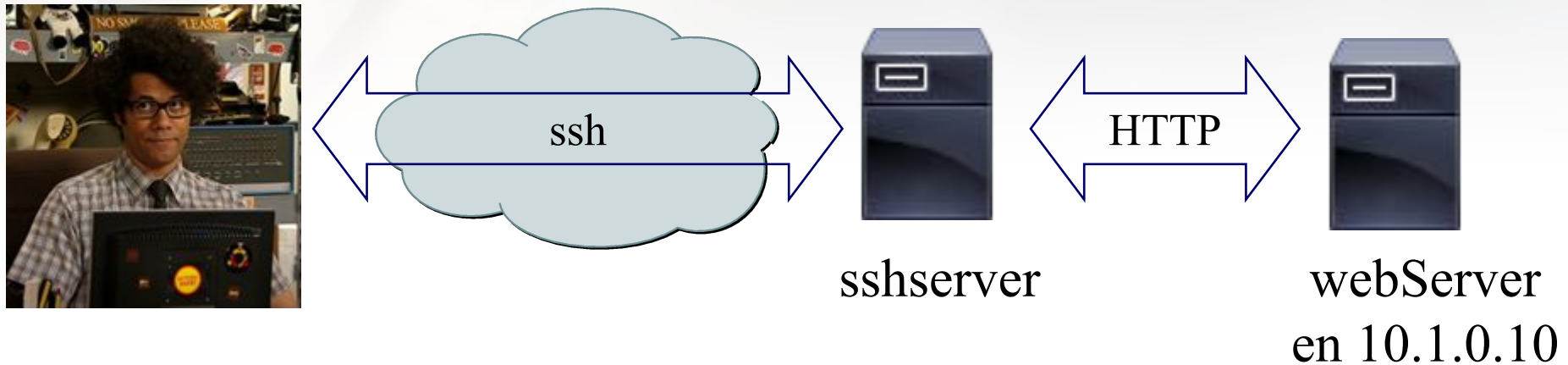
Moss quiere conectarse a un servidor SMTP remoto pero por alguna razón no puede o no quiere hacerlo directamente



SSH

Local Port forwarding: ejemplo

Moss accede a un servidor web que está "escondido" en una red privada.
Pero tenemos acceso a un servidor SSH dentro de esa red.



Paso 1: establecer la conexión

```
moss@reynholm~$ ssh -L2001:10.1.0.10:80 -l user sshserver
```

Se establece una sesión remota con el servidor *sshserver* pero además hará que todo lo enviado al puerto local 2001 sea enviado a través de la sesión SSH al puerto 80 de *webServer*.

Local Port forwarding: ejemplo

Para acceder a un recurso del webserver podemos hacer

```
moss@reynholm~$ curl localhost:2001 .....
```

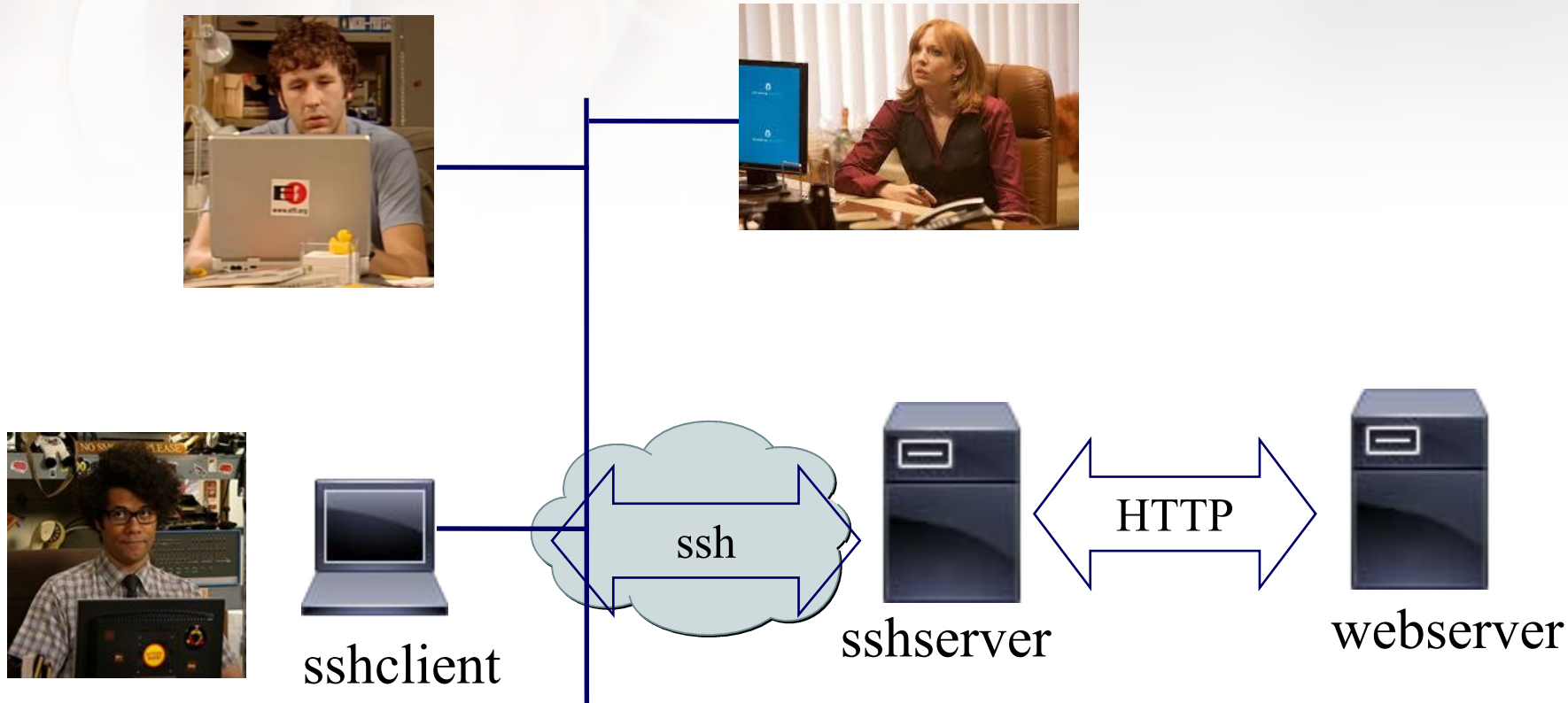
1. curl se conecta con TCP a (localhost,2001)
2. envía los requests
3. El cliente local SSH lee el puerto 2001, encripta los datos, y los envía al servidor SSH.
4. El servidor SSH desencripta los datos y los envía a 10.1.0.10:80.
5. Las respuestas son enviadas por el mismo túnel.

¿Qué debería cambiar si el servidor web funciona en el mismo host que el servidor SSH?

Para el web server ¿quién hizo el request?

Local Port forwarding: gateway

Los otros hosts de su red también quieren usar un túnel seguro al server remoto



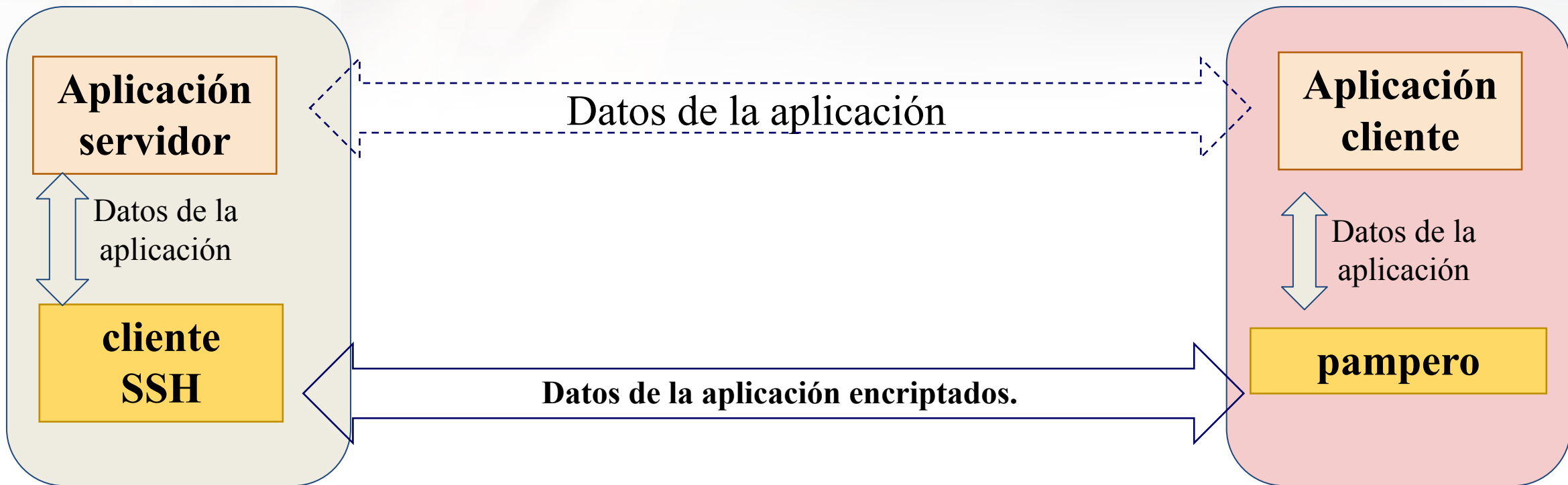
```
sshclient:~$ ssh -g -L2001:webserver:80 -l user sshserver
```

```
sshclient:~$ ssh -L 0.0.0.0:2001:webserver:80 -l user sshserver
```

SSH

Remote Port forwarding

`curl pampero:9999`

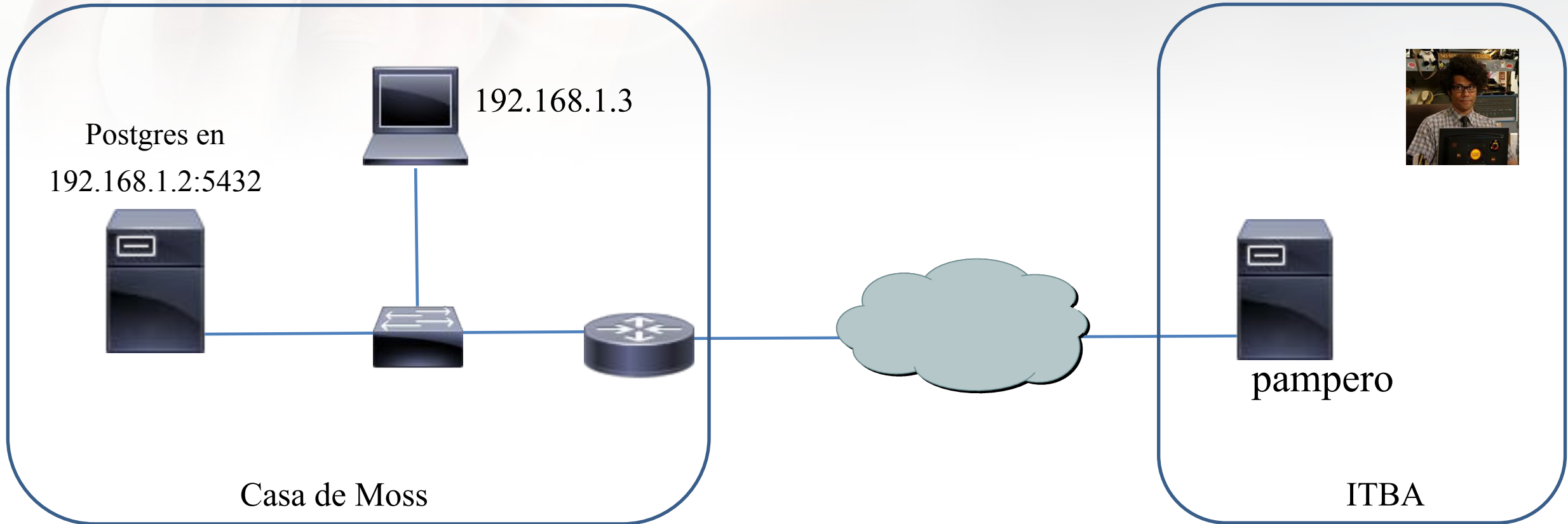


```
$ ssh -R9999:webserver:80 pampero.itba.edu.ar
```

SSH

Remote Port forwarding: ejemplo

Moss tiene en su casa un servidor Postgres y quiere conectarse desde el ITBA.

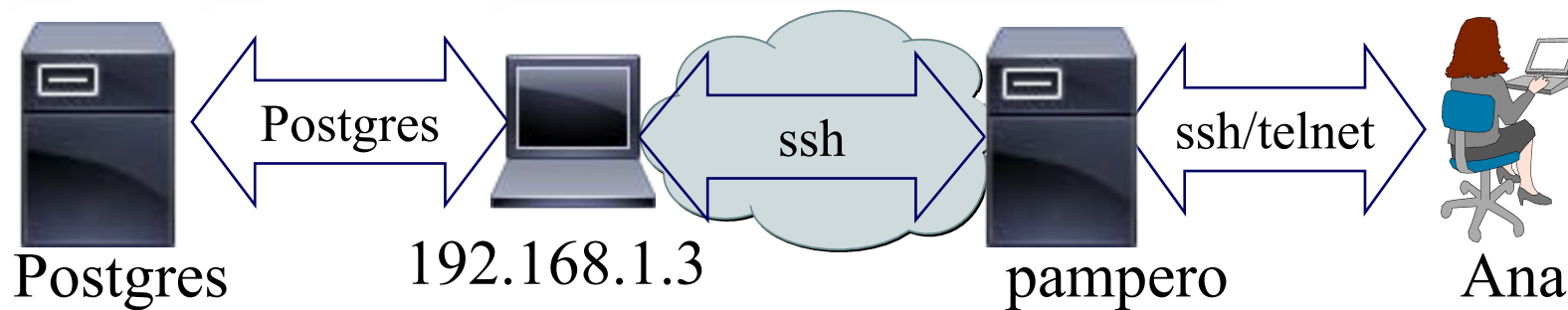


```
192.168.1.3:~$ $ ssh -R2001:192.168.1.2:5432 moss@pampero
```

SSH

Port forwarding remoto

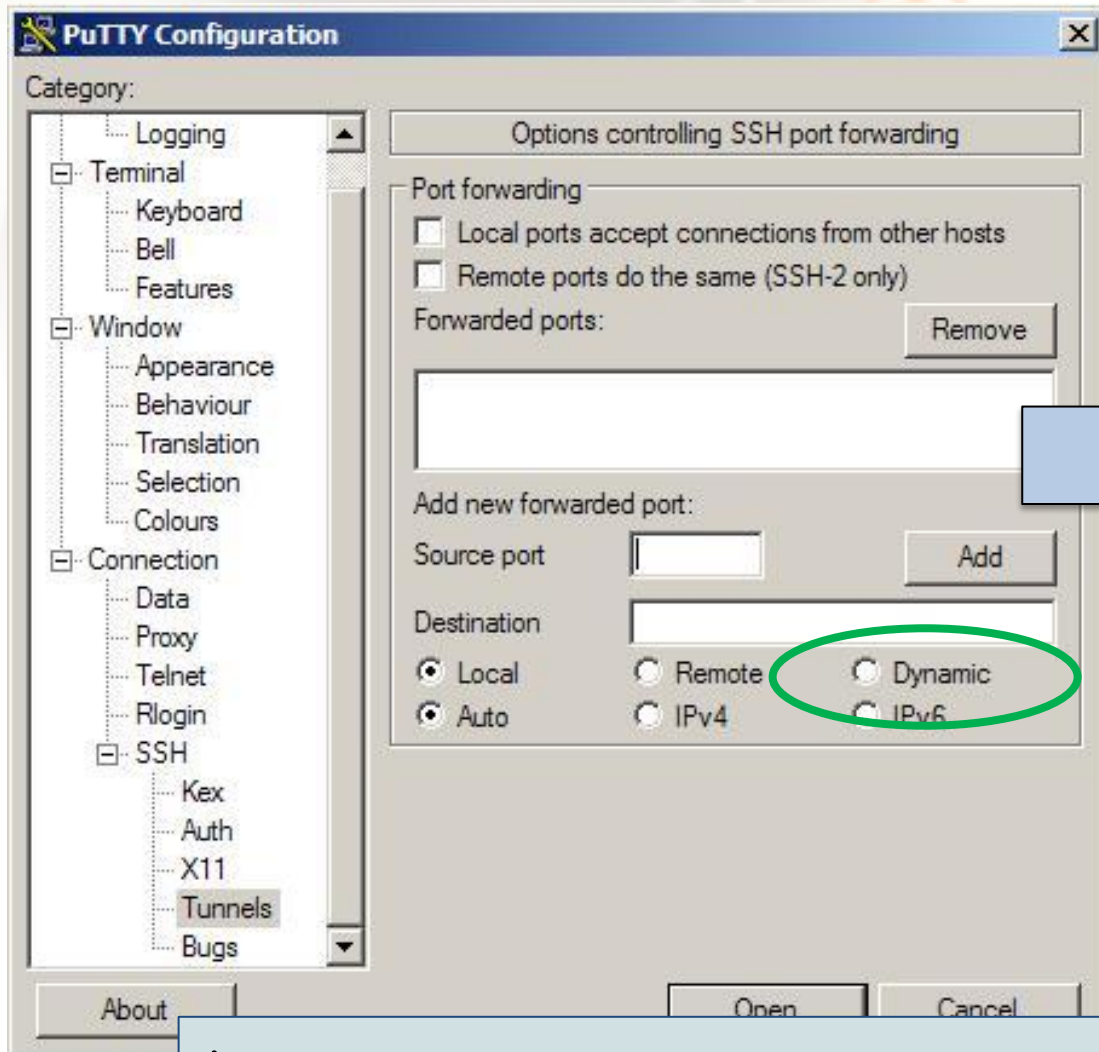
El puerto remoto será el 2001, el puerto local el 5432.
Una vez establecida la sesión, se ha creado un túnel desde el puerto 2001 del host remoto al puerto 5432 en el servidor Postgres.



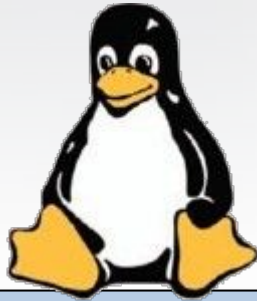
```
ana@pampero:~$ psql -p 2001
```

**Por defecto todos los usuarios en
pampero podrán usar el túnel**

Dynamic Port forwarding



```
$ ssh -D 9090 pampero.itba.edu.ar
```



```
$ google-chrome --proxy-server="socks5://localhost:9090"
```


Autenticación

Existen tres formas de autenticar un cliente SSH:

- Claves
- Claves públicas
- Basada en el host

Autenticación por claves

Es la forma más común y controla que el nombre de usuario y clave sean válidos, generalmente usando el archivo `/etc/passwd` o `/etc/shadow`.

Para deshabilitar este método editar `/etc/ssh/sshd_config`

```
PasswordAuthentication no
```

Autenticación

Claves públicas

Requiere que cada usuario genere un archivo con su clave pública y otro con su clave privada.

La clave pública debe ser copiada al servidor y la clave privada debe estar en el host cliente.

En el archivo `sshd_config` debe figurar

```
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys
```

Autenticación

Pasos a seguir en el host cliente Linux

```
user@server:~$ ssh-keygen -d
Generating public/private dsa key pair.
Enter file in which to save the key
    (/home/.../.ssh/id_dsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in id_dsa
Your public key has been saved in id_ds.pub
```

Ahora el usuario dispone de archivos con clave pública y clave privada. Debe conservar la clave privada en el o los hosts origen y almacenar la clave pública en el o los hosts destino.

Autenticación

Pasos a seguir en el host servidor: **Copiar el archivo .pub**

```
user@server:~$ cd
user@server:~$ cat id_dsa.pub >>.ssh/authorized_keys
user@server:~$ rm id_dsa.pub
```


Autenticación

Usar clave pública con Putty y OpenSSH

1. Ejecutar el programa puttygen para generar las claves en el host cliente
2. Indicar en Putty, dentro de la opción Connection / SSH / Auth el archivo que contiene la clave privada
3. Agregar la clave en el servidor, de la misma forma que en el ejemplo anterior.

En caso de poseer archivos de clave privada generada por ejemplo con OpenSSH se puede utilizar puttygen para importar dicha clave.

Autenticación

El cliente debe autenticar al servidor (*Server authentication*).



Soy Moss (*primera conexión*)

Clave pública de pampero



Pampero



Soy Moss

mensaje cifrado con clave privada pampero



Modelo de capas SSH



Modelo de capas SSH

- ◆ Transporte (SSH Transport Layer Protocol)
 - ◆ Provee autenticación de servidor, privacidad e integridad
 - ◆ Opcionalmente puede comprimir datos
 - ◆ Utiliza los servicios de TCP
- ◆ Autenticación (SSH User Authentication Protocol)
 - ◆ Provee autenticación del lado del cliente
- ◆ Conexión (SSH Connection Protocol)
 - ◆ Se encarga de multiplexar el túnel seguro en los distintos canales lógicos
 - ◆ Algunos canales lógicos posibles
 - ◆ sesiones de shell seguras
 - ◆ TCP port forwarding
 - ◆ Conexiones X11

Conexión SSH

cliente
SSH

servidor
SSH

conexión TCP

intercambio de string de versión SSH

intercambio de claves SSH
(incluye negociación de algoritmo)

intercambio de datos SSH

fin de conexión TCP

Material de lectura

https://www.climagic.org/tutorials/SSH_Tutorial_for_Linux.php